

Tema 7 - Análisis Semántico

Profesor Ramón López-Cózar Delgado, Curso 21-22

Función del Analizador Semántico:

- Desde el punto de vista del analizador semántico, lo único que se necesita identificar es que determinadas palabras, pertenecientes a un componente sintáctico, deben aparecer precedidas o seguidas por otras palabras.
- El analizador debe:
 - Revisar el texto fuente para tratar de encontrar errores semánticos.
 - Reúne información de tipos necesaria para las fases posteriores de análisis.
 - Verifica que cada operador tenga operandos permitidos, por ejemplo, números enteros, números reales, matrices, cadenas, etc.
 - Realiza otras comprobaciones semánticas.
 - Identificar los lexemas de los que se compone cada token.

Función de la Tabla de Símbolos

- Un compilador usa la tabla de símbolos para llevar un registro de la información sobre el ámbito de los nombres: variables y símbolos
- Se examina la tabla de símbolos cada vez que se encuentra un nombre en el texto fuente.
- Un mecanismo de la tabla de símbolos debe permitir añadir entradas nuevas y encontrar las entradas existentes eficientemente.

Características de la Tabla de Símbolos

- Es una estructura volátil y dinámica
 - Dinámica en el sentido que va variando de tamaño en las fases de análisis.
 - Volátil porque solo se crea para compilar y luego se destruye.
- Simplifica el análisis sintáctico.
- Ayuda en la comprobaciones semánticas.
- Ayuda en la generación de código.

Contenido de la Tabla de Símbolos

- Esencialmente la información que aparece en la tabla de símbolos es de dos tipos:
 - El propio símbolo, es decir, el propio nombre de la variable o procedimiento.
 - Atributos necesarios para definir el símbolo a nivel semántico y de generación de código (variables, subprogramas, etiquetas, etc.)
- Los atributos requeridos para cada símbolo dependen a nivel general de:
 - Del tipo de gestión de memoria (dirección, ...)
 - Si el lenguaje está o no estructurado en bloques.
 - Si el símbolo es, o no, parámetro de un procedimiento o función.
- El analizador léxico deberá:
 - Crear la tabla de símbolos parcialmente.
 - Insertar los símbolos detectados en la Tabla de Símbolos.
 - Señalar la línea del programa fuente en donde aparecen.
- El analizador semántico deberá:
 - Añadir los tipos, si procede, a los símbolos que aparecen en la tabla de símbolos.

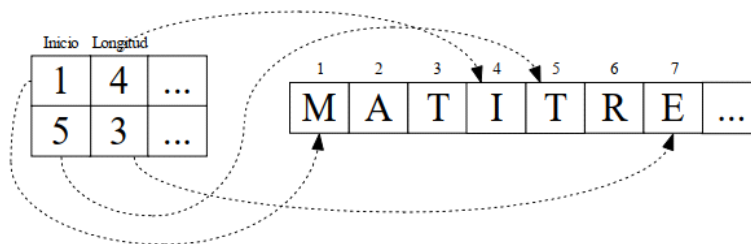
Operaciones con la Tabla de Símbolos

- Pueden ser:
 - *Crear* la tabla de símbolos.
 - *Insertar* nombres de variables, etiquetas y procedimientos.
 - *Consultar* dichos símbolos.
 - *Modificar*, es decir, añadir atributos nuevos a las entradas.

- *Crear subtablas* para cada bloque, si el lenguaje lo soporta.
- Cuándo y cómo se usan estas operaciones depende del tipo de lenguaje.
 - Declaración de variables de forma explícita
 - Declaraciones: solo *Insertar*.
 - Referencia: solo *Consultar*.
 - Declaración de variables de forma implícita
 - *Consultar* si está ya incluida.
 - *Insertar* en caso contrario.
 - Lenguajes con estructura de bloques
 - *Crear subtablas*.

Estructura de la Tabla de Símbolos

- La distribución de la información de la Tabla de Símbolos dependerá de las características del lenguaje y de las restricciones establecidas para los símbolos.
- **Campo dedicado para el símbolo**
 - Formato fijo:
 - Apropiado cuando se establece un límite en el número de caracteres que forman los símbolos, y además, sea pequeño.
 - En este caso sólo se dispone de un área fija en la tabla para almacenar el símbolo.
 - Formato variable:
 - Se dispone de la Tabla de Símbolos y de un área auxiliar en donde se introducen los símbolos de modo consecutivo, en la Tabla de Símbolos se sustituye el campo dedicado para el nombre del símbolo por un puntero al área auxiliar y un entero que indica la longitud del mismo.



- **Campo de Dirección**
 - En lenguajes sin estructuras de bloques, se asignan direcciones consecutivas según el orden en el que aparecen declaradas.
 - En lenguajes con estructuras de bloques, para cada bloque se asigna una subtabla: la dirección será consecutiva para cada bloque.
 - Se necesitan dos campos:

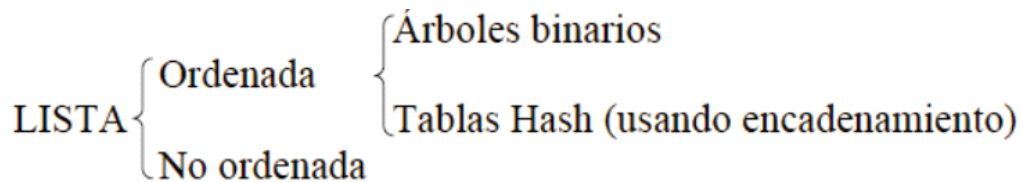
Nº Bloque	Dirección Bloque
-----------	------------------

Se introduce este campo en la Tabla de Símbolos cuando se declara, se utiliza este atributo en la fase de generación de código.

- **Campo de tipo**
 - Se introduce cuando se identifica una declaración explícita o implícita de una variable.
 - Se utiliza para determinar la memoria de almacenamiento y para la comprobación de tipos.
- **Campo de N.º de dimensiones / N.º de parámetros**
 - La realiza el analizador semántico y sirve para delimitar el tamaño de memoria necesaria para representar el símbolo.

- Por ejemplo, el tamaño o dimensión de un array o si se trata de una función o procedimiento, es el número de argumentos que posee y sus tipos para la reserva de memoria.
- **Campo de Lista Cruzada de Referencia**
 - Indica los lugares de un programa donde se hace referencia a una variable.
- **Campo de Puntero de Orden**
 - Es el puntero a donde aparece una variable.

Organización en lenguajes SIN estructura de bloques



Organización en lenguajes CON estructura de bloques

- La tabla se estructura como una pila cuyos elementos son las subtablas asociadas con los bloques.
- Además, se añade una tabla ÍNDICE que apunta al inicio de cada subtabla, indicando el comienzo y el final de cada bloque.
- Para el manejo de la estructura tipo pila se añaden dos operaciones:
 - Set: Para marcar el comienzo de cada bloque.
 - Reset: Para marcar el final de bloque.

Gramática con Atributos:

- Una gramática con atributos es una gramática que:
 - A los símbolos terminales y no terminales se les pueden asociar atributos, que deberán tomar valores en un dominio predeterminado.
 - A las producciones de la gramática se les puede asociar reglas de evaluación de los atributos asociados con los símbolos que aparecen en la producción.
 - Las reglas de evaluación asociadas con una producción son ejecutadas cuando la producción sea aplicada en el proceso de análisis.
- Matemáticamente, sea $G = (T, N, P, S)$ una gramática libre de contexto y $C = \{c_1, c_2, \dots, c_n\}$ el conjunto de atributos asociados con los símbolos de la gramática G .
 - Dada una regla de evaluación $b = f(c_1, \dots, c_k)$ asociada con la producción $A \rightarrow \alpha \in P$
 - **Atributo heredado:** Si b está asociado con algún símbolo de α .
 - **Atributo sintetizado:** Si b está asociado con el símbolo no terminal A .

Atributos Sintetizados

- **Evaluación:**
 - Las reglas de evaluación de los atributos sintetizados se suelen ejecutar cuando se realizan reducciones en el análisis sintáctico ascendente.
- **Requisitos para la evaluación:**
 - Las reglas de evaluación de los atributos sintetizados deben definirse en función de los atributos asociados con los símbolos gramaticales a su derecha.
- **Gramática S-Atributida:**
 - Una gramática es S-Atributida cuando todos sus atributos son sintetizados.
 - Se desea que la gramática sea así y es lo que hace uso YACC.
 - La estructura de pila se adecua para que cada símbolo de la gramática disponga de sus atributos asociados.

Atributos Heredados

- La evaluación de un atributo heredado depende de los atributos asociados con los símbolos

precedentes en la derivación.

- Los atributos heredados se suelen evaluar cuando se realizan derivaciones en el análisis sintáctico descendiente.
- **Gramática L-Atribuida:**
 - Sea la producción $A \rightarrow X_1, X_2, \dots, X_n$, una gramática es L-Atribuida si todos los atributos heredados asociados con X_j , $1 \leq j \leq n$, solo dependen de:
 - Los atributos asociados con los símbolos $X_1, X_2, \dots, X_{j-1}$
 - Los atributos asociados con A .
 - Es decir, los atributos de un símbolo en particular depende solamente de los atributos de los símbolos que tiene a su izquierda.
 - Toda gramática L-Atribuida puede ser evaluada mediante análisis sintáctico ascendente.

Evaluación de los atributos

- **Árbol de análisis**, para cada entrada se construye el árbol sintáctico (grafos acíclicos).
- **Basado en reglas semánticas**, dependiendo de las reglas semánticas (atributos heredados o sintetizados) se establece el orden de evaluación.
- **Dirigido por sintaxis**, el orden de evaluación es impuesto por la estrategia de análisis.

Esquemas de Traducción

- Son gramáticas con atributos en que las acciones semánticas se insertan en las producciones entre $\{ \}$.
- El lugar donde se insertan las acciones semánticas indica el lugar donde se pueden evaluar los atributos
 - Un atributo sintetizado del símbolo no terminal a la izquierda de una producción solo se puede evaluar cuando se han evaluado los atributos de los que depende, la acción usualmente se sitúa al final de la producción aunque también puede aparecer entre símbolos.
 - Una acción no debe referir un atributo sintetizado de un símbolo que esté en la derecha de la producción.
 - Un atributo heredado de un símbolo del lado derecho de la producción se debe evaluar en una acción que aparezca antes que el símbolo.

Comprobaciones Semánticas:

- **Comprobaciones estáticas:**
 - Sintácticas y semánticas
- **Comprobaciones dinámicas:**
 - Realizadas en tiempo de ejecución.
- Se tienen comprobaciones de:
 - **Tipo:** Verificación del tipo de los operandos en las expresiones.
 - **Flujo de Control:** Verificación de los puntos del programa de salida y entrada de control.
 - **Unicidad:** Verificación de la presencia de símbolos de forma única, por ejemplo, que solo se pueda declarar un símbolo una vez.
 - **Relación de nombres:** Un mismo nombre puede aparecer más de una vez, por ejemplo, un nombre de bloque en Ada.

Expresiones de Tipos

- Los tipos poseen una estructura la cual se representan por medio de expresiones de tipos
 - Los tipos básicos o simples son expresiones de tipo, por ejemplo: *int*, *float*, *bool*, *double*, ...
 - El propio nombre de un tipo es una expresión de tipo.
 - Se forman también mediante la aplicación de un operador llamado constructor de tipos a una expresión de tipos.
- En una expresión de tipos pueden aparecer variables de tipos.

Constructores de Tipos

- **Matrices (Arrays)**
 - $\text{Array}(I, T)$ es una expresión de tipo que representa el tipo de una matriz con elementos de tipo T y conjunto de índices I .

En Pascal:

```
VAR A: array [1..10] of integer;
```

Le asocia la expresión tipo: $\text{array}(1..10, \text{integer})$ a A

- **Producto**
 - Sea T_1 y T_2 expresiones de tipo, entonces $T_1 \times T_2$ (producto cartesiano) es también una expresión de tipo.
- **Registros**
 - Como un producto pero con nombre

Record ((dirección × integer) × (lexema × array(1..15,char)))	
PASCAL	Lenguaje C
<pre>type file=record direccion : integer ; lexema : array[1..15] of char ; end ;</pre>	<pre>typedef struct { int direccion ; char lexema[15] ; } file ;</pre>

- **Apuntadores**
 - Sea T una expresión de tipo, entonces $\text{pointer}(T)$ es una expresión de tipo.
 - Por ejemplo, $\text{pointer}(\text{FILA})$ apunta a un objeto FILA
- **Funciones**
 - Como una función matemática, sean D y R dos expresiones de tipo, entonces $D \rightarrow R$ es otra expresión de tipo.

$\text{char} \times \text{char} \rightarrow \text{pointer}(\text{integer})$

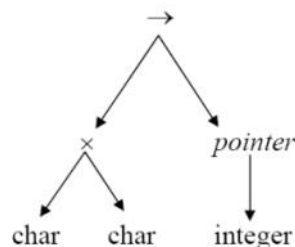


```
function f(a, b : char) : ^integer;
```

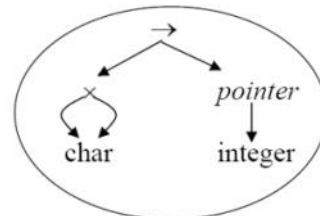
Representación de las expresiones de tipos

- Mediante un grafo dirigido acíclico (GDA)

Ejemplo 7.5: Representación de la expresión de tipo $\text{char} \times \text{char} \rightarrow \text{pointer}(\text{integer})$



hojas interiores: constructor de expresiones
o expresiones de tipos.
hojas finales: tipos básicos.



- Mediante mapas de bits, como se utiliza en el compilador de C; se parten de constructores básicos.
 - La principal ventaja es que se ahorra espacio, dos secuencias de bits distintas representan expresiones de tipos distintos.
 - Aunque tipos distintos podrían tener la misma secuencia de bits.

Codificación

pointer (T) : Expresión Puntero → 01
freturn (T) : Expresión función → 11
array (T) : Expresión array → 10

<u>Tipos básicos</u>	<u>Codificación</u>
Boolean	→ 0000
Char	→ 0001
Integer	→ 0010
Real	→ 0011

<u>Expresiones de Tipo</u>	<u>Mapa de Bit</u>
<i>char</i>	000000 0001
<i>freturn</i> (<i>char</i>)	000011 0001
<i>pointer</i> (<i>freturn</i> (<i>char</i>))	000111 0001
<i>array</i> (<i>pointer</i> (<i>freturn</i> (<i>char</i>)))	100111 0001

Equivalencia de Tipos

- **Equivalencia estructural:** Dos expresiones de tipos son equivalentes estructuralmente si responden a un mismo tipo básico o están formadas en base de aplicar el mismo constructor de tipos sobre expresiones equivalentes estructuralmente o sobre los mismos tipos básicos.
 - Para determinar si dos expresiones son equivalentes:
 - Se aplica GDA
 - Se aplica el algoritmo de análisis de los constructores.
 - El algoritmo podría ser algo como

```

function equivest (s, t) : boolean ;
begin
  if s y t son el mismo tipo_básico then
    return true ;
  else if s = array(s1,s2) and t = array(t1,t2) then
    return equivest (s1,t1) and equivest (s2,t2);
  else if s = s1×s2 and t = t1×t2 then
    return equivest (s1,t1) and equivest (s2,t2);
  else if s = pointer(s1) and t = pointer(t1) then
    return equivest (s1,t1);
  else if s = s1→s2 and t = t1→t2 then
    return equivest (s1,t1) and equivest (s2,t2)
  else
    return false ;
end;

```

- **Equivalencia de nombre:** Cuando se da nombre a las expresiones de tipos, éstas son nombradas, entonces dos expresiones de tipos son equivalentes de nombre si y solo si ambas expresiones son idénticas.
 - Si aparecen expresiones de tipos a las que se ha dado nombre, también ha de ser reconsiderado el concepto de equivalencia estructural.
 - Dos expresiones de tipo son equivalentes estructuralmente si sustituidos los nombres de tipos por sus correspondientes expresiones de tipos, resultan dos nuevas expresiones de tipos que son equivalentes estructuralmente.

Comprobación de tipos

- Comprobador de tipos elementales:
 - Un comprobador ha de disponer de:
 - Asignación de tipos.
 - Comprobador de tipos en las expresiones.
 - Comprobador de tipos en las proposiciones o sentencias.
 - Comprobador de tipos de las funciones.
 - Se advierten dos tareas:
 - De asignación
 - De evaluación y comprobación.

Ejemplo 7.7: Sea un lenguaje sencillo que obedece a una gramática con el conjunto de producciones siguientes.

$$\begin{aligned}
 P &\rightarrow D ; E ; S \\
 D &\rightarrow D ; D \\
 D &\rightarrow id : T \mid \varepsilon \\
 T &\rightarrow \text{char} \mid \text{integer} \mid \text{array [num] of } T \mid ^T \mid T \rightarrow T \\
 E &\rightarrow \text{literal} \mid \text{num} \mid id \mid E \text{ mod } E \mid E[E] \mid E^{\wedge} \mid E(E) \mid \varepsilon \\
 S &\rightarrow id := E \mid \text{if } E \text{ then } S \mid \text{while } E \text{ do } S \mid S ; S
 \end{aligned}$$

Las comprobaciones semánticas vendrán referidas a:

- (a) Asignación de tipos.
- (b) Comprobación de tipos en expresiones.
- (c) Comprobación de tipos en proposiciones.
- (d) Comprobación de tipos en funciones.

(a) Asignación de tipos:

$$P \rightarrow D ; E ; S$$

$$D \rightarrow D ; D$$

$$D \rightarrow id : T$$

$$\{ \text{añadetipo}(id.entrada, T.tipo); \}$$

$$T \rightarrow \text{char}$$

$$\{ T.tipo := \text{char}; \}$$

$$T \rightarrow \text{integer}$$

$$\{ T.tipo := \text{integer}; \}$$

$$T \rightarrow ^T$$

$$\{ T.tipo := \text{pointer}(T.tipo); \}$$

$$T \rightarrow \text{array[num] of } T_1$$

$$\{ T.tipo := \text{array}(1..num.val, T_1.tipo); \}$$

$$T \rightarrow T_1 \rightarrow T_2$$

$$\{ T.tipo := T_1.tipo \rightarrow T_2.tipo; \}$$

(b) Comprobación de tipos en expresiones:

$$E \rightarrow \text{literal}$$

$$\{ E.tipo := \text{char}; \}$$

$$E \rightarrow \text{num}$$

$$\{ E.tipo := \text{integer}; \}$$

$$E \rightarrow id$$

$$\{ E.tipo := \text{busca}(id.entrada); \}$$

$$E \rightarrow E_1 \text{ mod } E_2$$

$$\{ E.tipo := \text{if } E_1.tipo = \text{integer} \text{ and } E_2.tipo = \text{integer} \\ \text{then integer} \\ \text{else error_tipo}; \}$$

$$E \rightarrow E_1[E_2]$$

$$\{ E.tipo := \text{if } E_2.tipo = \text{integer} \text{ and } E_1.tipo = \text{array}(s, t) \\ \text{then } t \\ \text{else error_tipo}; \}$$

$$E \rightarrow E_1^{\wedge}$$

$$\{ E.tipo := \text{if } E_1.tipo = \text{pointer}(t) \\ \text{then } t \\ \text{else error_tipo}; \}$$

(c) Comprobación de tipos en proposiciones:

$S \rightarrow \text{id} := E$	{ id.tipo := <i>busca</i> (id.entrada); S.tipo := if id.tipo=E.tipo then vacio else error_tipo; }
$S \rightarrow \text{if } E \text{ then } S_1$	{ S.tipo := if E.tipo=boolean then S ₁ .tipo else error_tipo; }
$S \rightarrow \text{while } E \text{ do } S_1$	{ S.tipo := if E.tipo=boolean then S ₁ .tipo else error_tipo; }
$S \rightarrow S_1 ; S_2$	{ S.tipo := if S ₁ .tipo=vacio and S ₂ .tipo=vacio then vacio else error_tipo; }

(d) Comprobación de tipos en funciones: En la fase de declaraciones ha de ser definido el tipo.

$E \rightarrow E_1(E_2)$	{ E.tipo := if E ₂ .tipo= <i>s</i> and E ₁ .tipo= <i>s</i> → <i>t</i> then <i>t</i> else error_tipo; }
--------------------------	--

Tópicos en la comprobación de tipos:

Conversión de tipos (coerción)

- **Implícita:** Se realiza automáticamente por medio del compilador.
- **Explícita:** El programador indica expresamente la conversión.
 - Ejemplo de coerción implícita

$E \rightarrow \text{ConstEntera}$	{ E.tipo := integer ; }
$E \rightarrow \text{ConstReal}$	{ E.tipo := real ; }
$E \rightarrow \text{id}$	{ E.tipo := <i>busca</i> (id.entrada) ; }
$E \rightarrow E_1 \text{ op } E_2$	{ E.tipo := if E ₁ .tipo=integer and E ₂ .tipo=integer then integer else if E ₁ .tipo=integer and E ₂ .tipo=real then real else if E ₁ .tipo=real and E ₂ .tipo=integer then real else if E ₁ .tipo=real and E ₂ .tipo=real then real else error_tipo ; }

Sobrecarga de funciones y operadores

- +, − son símbolos sobrecargados porque su significado dependerá del contexto, unarios y binarios.
- (,) pueden estar sobrecargados en lenguajes como ADA y FORTRAN ya que se usan como referencia de elementos de matrices y como funciones, la sobrecarga se resuelve en algunos casos en base a los tipos de las expresiones en donde aparecen, se le conoce como estudio de contexto.